Cybersecurity Practical Assignment

Secure Application Security Assessment & Implementation

Practical Technical Assessment

1. Objective

The objective of this assignment is to assess your practical cybersecurity skills by implementing security controls on a small, working web application.

You will not be provided access to any production system, internal infrastructure, database, network, source code, or operational environment. You must create your own isolated application/environment and use it to demonstrate the required security controls.

2. Application to Be Used for the Assessment

Build a small working web application of your choice. The application itself is not the primary focus; it is the platform on which you will demonstrate your cybersecurity skills.

Examples include:

- Employee management system
- Energy monitoring dashboard
- Document management system
- Inventory management system
- Any other small web application approved by the assessment team

The application must be functional enough to demonstrate authentication, APIs, user roles, data access, logging, and security testing. A mock interface or static demonstration is not sufficient.

3. Mandatory Security Requirements

3.1 Secure Authentication & Access Control

Implement a secure authentication and authorization system with:

- Login and logout functionality.
- Role-Based Access Control (RBAC) with at least three roles.
- MFA for the administrator account.
- Account lockout or effective rate limiting after repeated failed login attempts.
- Short-lived access tokens.
- Secure refresh-token handling.
- Server-side authorization checks.
- No shared/default accounts and no hardcoded credentials.

Use RS256 asymmetric JWT authentication with an access-token lifetime of 15 minutes or less. Demonstrate that a user with a lower-privileged role cannot access administrator functionality by directly calling the API.

3.2 API & Application Security

Secure the application against common web application attacks. At minimum, demonstrate protection against:

- SQL Injection
- Cross-Site Scripting (XSS)
- Broken Access Control
- Brute-force login attempts
- Malicious or invalid input

The implementation should include:

- Server-side input validation.
- Parameterized queries or a secure ORM.
- Appropriate output encoding.
- Content Security Policy (CSP).
- HSTS.
- X-Frame-Options.
- X-Content-Type-Options.
- Generic error messages to users, with detailed errors retained only in server-side logs.

3.3 Network & Deployment Security

Deploy the application using a simple multi-tier architecture. A suggested structure is:

```
Internet → Reverse Proxy / WAF → Application / API → Database
```

Implement and demonstrate:

- Reverse proxy in front of the application.
- HTTPS for application traffic.
- Only required network ports exposed.
- Database not directly accessible from the internet.
- Appropriate firewall/network restrictions.
- Administrative access restricted appropriately.

Provide an architecture diagram and explain the purpose of each component and the allowed communication paths between components.

3.4 Security Logging & Attack Detection

Implement a backend security/audit logging mechanism. At minimum, record:

- Successful login
- Failed login
- Account lockout
- Unauthorized access attempts
- Role or permission changes
- Administrator activity
- Configuration changes
- API errors
- Data export or download events

Implement at least three meaningful security alerts. Examples include:

- Repeated failed login attempts from the same account or source.
- Repeated unauthorized/403 requests.
- Unusual or excessive data export.
- Repeated privilege/authorization failures.

Security events must be generated by the backend and stored in a manner that supports later investigation. Simply displaying hardcoded logs on a frontend is not acceptable.

3.5 DevSecOps Security Pipeline

Create a basic CI/CD security pipeline for the application. The pipeline must include:

- Static Application Security Testing (SAST)
- Software Composition / Dependency Analysis (SCA)
- Secret scanning
- Container image scanning
- Dynamic Application Security Testing (DAST)

Configure the pipeline so that High/Critical security findings can block the build or deployment. You may use appropriate open-source tools such as Semgrep, Gitleaks, Trivy, OWASP ZAP, Bandit, pip-audit, npm audit, or equivalent tools.

4. Security Testing & Demonstration

You are expected to test your own application and demonstrate that the implemented controls work. At minimum, demonstrate five security tests covering the areas implemented in this assignment.

For each test, document:

- The attack or security scenario.
- What could happen without the security control.
- The security control implemented.
- How the test was performed.
- The observed result.
- Evidence such as screenshots, logs, or tool output.

Examples:

- SQL Injection attempt and prevention.
- Viewer attempting to access an administrator endpoint.
- Repeated failed login attempts triggering rate limiting/lockout.
- XSS payload being rejected or safely encoded.
- Expired or revoked token being rejected.
- Security scan identifying and then confirming remediation of a vulnerability.

5. Security Report

Submit a concise technical report explaining the security design and implementation. The report should contain:

- System overview and architecture.
- Authentication, MFA and RBAC design.
- Application/API security controls.
- Network and deployment security.
- Logging and alerting design.
- DevSecOps security pipeline.
- Security testing performed and results.
- Security vulnerabilities identified and how they were remediated.
- Known limitations and recommendations for a production deployment.

6. Deliverables

- Complete source-code repository.
- README with setup, configuration, execution and testing instructions.
- Architecture diagram.
- Security report.
- CI/CD pipeline configuration.
- Evidence of security testing and scan results.
- Docker configuration or equivalent reproducible deployment configuration.

7. Important Submission Requirements

The assessment must contain a real working implementation.

- Mock screens without working backend security controls will not be considered an implementation.
- Hardcoded security responses or fake audit logs will not be accepted.
- Security controls must be implemented and demonstrable.
- The candidate should be able to explain all major security decisions and implementation details.
- All testing must be performed only against the candidate's own isolated environment.

8. Technology Choice

The technology stack is open. You may choose the programming language, framework, database, container platform, reverse proxy, security tools, and CI/CD platform. You must be able to explain the security implications of your choices.

9. Production Security Considerations

In your report, briefly explain how your solution would need to be adapted before being deployed in a real production environment handling sensitive or critical operational data. Address, at a minimum:

- Network segmentation and DMZ placement.
- Restricted administrative access.
- Centralized/tamper-evident logging and retention.
- Secure key and secret management.
- Backup and recovery.
- Security assessment/audit before production deployment.